Notes

- **Темата започва малко по-надолу**
- възможно е да има ненужни функции
- Всяка функции от имплементацията трябва да я опишете какво и как го прави!!!
- Всяка сложност трябва да бъде обоснована защо се получава!!!
- Темата така разгърната носи 6.00 (тествано)

Зададени въпроси към преподавател

1. На пета точка за имплементация, какво точно трябва да накодя за кореново дърво? Не мога да разбера какъв точно трябва да бъде интерфейса на структурата?
2. Нужно ли е да пиша Move semantics в имплементациите?
3. Грешките по член функциите в имплементациите трябва ли да са по-специфични от std::exception?
4. С какви exception safety guarantees трябва да ми бъдат имплементациите?
5. На точката за LinkedList трябва ли да се коментира имплементация, когато има поинтър към опашка и когато няма? Или се приема, че винаги говорим за имплементация с поинтър към опашката?
6. Трябва ли на BST да се обясни как работят insert, find, remove?
7. Има ли значение дали ще напиша статична, динамична или свързана имплементация на структурите (например опашката)? Понеже статичната е най-бърза за разписване, но това ще се отрази ли на точките ми?

Отговори:

1. За кореново дърво обикновено обяснявате какво точно дърво реализирате (например директорийна структура), показвате операции по добавяне (по някакви критерии), съответно търсене (ако има смисъл) и обхождания (ширина, дълбочина).
2. Не
3. Не е нужно да имате изключения - може да връщате bool като резултат или друг избран от вас начин за сигнализиране на грешките
4. Достатъчно е да гарантирате коректно състояние и да няма течове на ресурси (weak guarantee).
5. Няма значение, стига да сте описали какво и защо правите (и да сте съобразили функциите, които предлагате с избора). Например не е ОК да нямате указател към края, но да посочите като удобна функционалност добавяне в двата края.
6. За всяка структура освен кода трябва да имате и обяснения към него. В частност и за BST
7. Няма значение, стига да сте описали добре нещата.

Тема 15. Структури от данни. Стек, опашка, списък, дърво. Основни операции върху тях. Реализация.

Имплементациите са в папка Data-Structures на root ниво

При разписване на темата трябва да опишете как работят съответните основни функции на структурите ви (т.е. тези от интерфейса).

1. Структури от данни - дефиниране на понятието

Note: Тази точка не е от особено значение на фона на другите, затова всичко е дефинирано възможно най-просто в нея.

Структура от данни

- Структура от данни - организиране на данни в паметта, което позволява ефективно съхранение и достъп до тях.
- Абстрактна тип данни (Abstract data type) - Концептуален модел, който дефинира множество от операции и поведение на структура от данни, без да специфицира как те трябва да бъдат имплементирани и как данните им да бъдат представени в паметта. Това позволява един тип абстрактни данни да има различни реализации и респективно различна ефективност.

Видове структури от данни:

- Според вида на компонентите:
  - Хомогенни - всички елементи са от един и същи тип (array)
  - Хетерогенни - елементите могат да бъдат от различни типове (struct)
- Според способността за промяна на размера
  - статични
  - динамични
- Според връзките между елементите:
  - линейни
  - нелинейни
- Според начина на достъп до елементите:
  - последователни
  - директни

// **Note:** Алгоритъм - не е записано в темата, но според мен трябва да се обясни с две думи, понеже надолу в темата се използват сложности за операциите при структурите // все пак не е възможно и да се опише всичко подробно за два часа

Алгоритъм

- Алгоритъм - последователност от стъпки, които се изпълняват за решаване на даден проблем.

Сложности

- Сложност - мярка за количеството на ресурси (време, памет), които алгоритъмът изисква за изпълнение.

Видове сложности:

- Сложност на алгоритъм по време - описва как броят на елементарните операции, необходими за изпълнение на алгоритъма, нараства спрямо размера на входа
- Сложност на алгоритъм по памет - описва как количеството памет, необходимо за изпълнение на алгоритъма, нараства спрямо размера на входа
- Как измерзваме сложността:
    - Worst Case - необходими ресурси в най-лош случай - това ще разглеждаме в рамките на темата
    - Average Case - в средния случай
    - Best Case - в най-добрия случай

Worst case се изразява с Big O нотация, която показва горна граница на колко бързо нараства асимптотата на една функция. Няколко различни функции с една и съща скорост на нарастване на асимпотата ще бъдат представени с едни и съща Big O нотация.

т.е. в O(n) влизат всички функции, чиято асимптота не нараства по-бързо от O(n) - 2n, 3n + 1 и други.

Основни сложности:

- O(1) - константна сложност
- O(1*) - амортизирана константна сложност
- O(log n) - логаритмична сложност
- O(n) - линейна сложност
- O(n log n) - линейно логаритмична сложност
- O(n^2) - квадратична сложност
- O(2^n) - експоненциална сложност
- O(n!) - факториална сложност

2. Списък

Логическо описание

- Линейна структура от данни, чиито данни не са наредени последователно в паметта, а всеки елемент има указател към следващия.
- Полезен при много добавяния в начало/край
- Полезен при сливане на два списъка O(1)

- Не се заделя голяма памет предварително, а се заделя динамично, когато е необходимо. За сметка на това има заделени допълнителни поинтъри към всеки елемент
- Не се възползва от locality (неформално: когато процесорът прочете нещо от паметта той кешира една идея по-голям блок в L_n cache-а, очаквайки че ще поискаш съседен блок памет скоро време)
- Търсенето е линейно
- няма директен достъп до елемент

Списък с една и две връзки

- **Note**: На тази точка няма значение дали ще изберете да опишете LL с указател за опашка. Важното е кода и описанията ви да съответстват.

Списък с една връзка

- Всеки елемента има указател към следващия елемент

Елемент на списъка се нарича Node (възел) и съдържа:

```
template<typename T>
struct Node {
    T data; // данните, които съхранява елемента
    Node<T>* next; // указател към следващия елемент
};
```

Списък с две връзки

- Всеки елемент има указател към следващ и предишен елемент

Елемент на списъка се нарича Node (възел) и съдържа:

```
template<typename T>
struct Node {
    T data; // данните, които съхранява елемента
    Node<T>* next; // указател към следващия елемент
    Node<T>* prev; // указател към предишния елемент
};
```

Интерфейс на списъка

```
class LinkedList {
private:
    struct Node {}; // зависимост какъв ви е node-а

    Node* head, * tail;
    size_t size = 0;
}
```

Характеристики на двете имплементации

Списък с една връзка

- Можем да обхождаме само в една посока

- един Node заема памет:
    - за данните (в зависимост от типа)
    - за указателя към следващия елемент

Списък с две връзки

- Можем да обхождаме в двете посоки
- един Node заема памет:
    - за данните (в зависимост от типа)
    - за указател към следващия елемент
    - за указател към предишния елемент

Сложност на операциите - добавяне, премахване, намиране на елемент

Операция	**Singly without tail**	**Singly with tail**	**Doubly**	**Double with tail**
Добавяне в началото	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Добавяне в края	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Намиране	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Премахване	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Премахване в началото	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Премахване в края	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$

Имплементация на двете реализация

В съответната папка.

3. Стек

Логическо описание.

Стек е абстрактен тип данни, който представлява колекция от елементи с три основни операции:

- push - която добавя елемент към колекцията
- pop - която премахва най-скоро добавения елемент.
- peek - връща стойността на последния добавен елемент

Редът, по който елементите се добавят или премахват от стека, се описва като „последен влязъл, първи излязъл“ (на английски: Last In, First Out, или съкратено LIFO).

Характеристики на статична, динамична и свързана реализация

- Статична реализация - използва масив с фиксиран размер, който не може да се променя по време на изпълнение. Ограничение е, че ако стекът е пълен, не може да се добавят нови елементи.
- Динамична реализация - използва динамично заделяне на памет, което позволява стекът да расте и се свива по време на изпълнение. Тоест ако текущо заделената памет се напълни елементи, стекът се разширява(като се заделя нова по-голяма памет), след което елементите се копират в новата памет, а старата се освобождава.
- Свързана реализация - използва свързани елементи (може да се използва вътрешно linked list), където всеки елемент има указател към следващия. Това позволява динамично добавяне и премахване на елементи, без да се налага да се заделя голяма памет предварително.

Сложност на операциите - добавяне, премахване, намиране на елемент.

Stack with	**Static Array**	**Dynamic Array**	**Linked list**
push()	$\mathcal{O}(1)$	Amortized $ \mathcal{O}(1)$	$\mathcal{O}(1)$
pop()	$\mathcal{O}(1)$	Amortized $ \mathcal{O}(1)$	$\mathcal{O}(1)$
peek()/top()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
clear()	$\mathcal{O}(1)$	$\mathcal{O}(p)$ - delete memory	$\mathcal{O}(n)$
isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
initialize	$\mathcal{O}(p)$ - на практика O(1	$\mathcal{O}(p)$ - allocate memory	$\mathcal{O}(1)$

Имплементация на трите реализации

В съответната папка.

4. Опашка

Логическо описание

Опашката е абстрактен тип данни, която представлява колекция от елементи с три основни операции:

- push - която добавя елемент към колекцията
- pop - която премахва най-отдавна добавения елемент.
- peek - връща стойността на най-отдавна добавения елемент

Редът, по който елементите се добавят или премахват от опашката, се описва като „първи

влязъл, първи излязъл“ (на английски: First In, First Out, или съкратено FIFO).

Характеристики на статична, динамична и свързана реализация

- Статична реализация - използва масив с фиксиран размер, който не може да се променя по време на изпълнение. Ограничение е, че ако опашката е пълна, не може да се добавят нови елементи.
- Динамична реализация - използва динамично заделяне на памет, което позволява опашката да расте и се свива по време на изпълнение. Тоест ако текущо заделената памет се напълни елементи, опашката се разширява(като се заделя нова по-голяма памет), след което елементите се копират в новата памет, а старата се освобождава.
- Свързана реализация - използва свързани елементи (може да се използва вътрешно linked list), където всеки елемент има указател към следващия. Това позволява динамично добавяне и премахване на елементи, без да се налага да се заделя голяма памет предварително.

Сложност на операциите - добавяне, премахване, намиране на елемент

Queue with	**Static Array**	**Dynamic Array**	**Linked list**
push()/enqueue()	$\mathcal{O}(1)$	Amortized $ \mathcal{O}(1)$	$\mathcal{O}(1)$
pop()/dequeue()	$\mathcal{O}(1)$	Amortized $ \mathcal{O}(1)$	$\mathcal{O}(1)$
peek()/top()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
clear()	$\mathcal{O}(1)$	$\mathcal{O}(p)$ - delete memory	$\mathcal{O}(n)$
isEmpty()	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
initialize	$\mathcal{O}(p)$ - на практика O(1)	$\mathcal{O}(p)$ - allocate memory	$\mathcal{O}(1)$

Имплементация на трите реализации

В съответната папка.

5. Дървовидни структури от данни - кореново дърво и двоично кореново дърво

Логическо описание

- Дървовидна структура от данни(Рекурсивна дефинирана), която се състои от възли, подобно на свързания списък. Един възел може да има няколко наследника, но само 1 предшественик.
- Нелинейна, рекурсивна, йерархична структура

- Разлика с граф - няма цикли
- Коренът на дървото е възелът без нито един предшественик.
- Ребро - връзка между двата върха
- Броят на възлите = Броят на ребрата + 1 (|V| = |E| + 1)
- Разклоненост - максималният брой върхове наследници, които може да има връх в едно дърво
- Път - поредица от ребра между върхове
- Листо - възел без наследници
- Дълбочина - дължината на пътя от корена до върха
- Височина на дърво - максималната дълбочина

Двоично дърво:

- Дърво, което има най-много два наследника

Кореново дърво:

- Дърво, което има избран конкретен връх за корен, който няма предшественици.

Начини на обхождане:

- Inorder (симетрично) - ляво, корен, дясно (валидно е и дясно, колрен, ляво, аналогично и за Post/Preorder)
- Preorder (префиксно) - корен, ляво, дясно
- Postorder (постфиксно) - ляво, дясно, корен
- Level order (по нива) - обхождане по нива, започвайки от корена и преминавайки през всички възли на текущото ниво, преди да преминем към следващото ниво.
- Балансиране - Дърво, за което височината на дървото се поддържа автоматично възможно най-малка спрямо броя на елементите в него с цел оптимални операции откъм търсене, премахване и добавяне. В случай на AVL балансираните дървета за всеки възел имаме свойството, че височината на лявото му поддърво се различава от височината на дясното поддърво с максимум единица.

Начини на представяне в паметта

Тъй като това е нелинейна структура, елементите не седят последователно в паметта. Всеки елемент има два указателя към други места в паметта, където са неговите деца. Представяне:

```
template<typename T>
struct Node {
    T value;
    Node* left = nullptr;
    Node* right = nullptr;
}
```

Имплементация на кореново дърво или двоично кореново дърво

- Имплементацията не е точно дефинирана. Т.е. трябва да си измислите нещо като ситуация, която имплементирате, но се абстрахирате от детайлите.

Пример:

Моят class BinaryTree представлява Файлова йерархия. Всеки елемент има стойност

- добавяне по критерий - попълваме първото празно място на най-горното възможно ниво
- търсене по стойност
- обхождания в ширина и дълбочина (няма нужда от всички наредби при обходане)

В съответната папка.

6. Двоично кореново дърво за търсене

NOTE: Доста от нещата от горната тема могат да бъдат споменани и тук.

Логическо описание

- Двоично дърво - Дърво, чиито възли имат най-много двама наследника. Най-често се обозначават ляв и десен (left, right).
- BST - Дефинира се рекурсивно по следния принцип:
    - Всяко листо е двоично дърво за търсене.
    - Всяко поддърво започващо от даден възел е двоично дърво за търсене, ако:
        - Най-големият елемент в лявото му поддърво е по-малък от стойността в дадения възел.
        - Най-малкият елемент в дясното му поддърво е по-голям от стойността в дадения възел. И лявото поддърво, и дясното поддърво са двоични дървета за търсене.
- При принтиране в реда ляво-корен-дясно ще получим всички елементи в нарастващ ред (съответно при дясно-корен-ляво в намаляващ)

Пример:

```
      8
     / \
    3   10
   / \    \
  1   6    14
     / \   /
    4   7 13
```

Начини на представяне в паметта

Тъй като това е нелинейна структура, елементите не седят последователно в паметта. Всеки елемент има два указателя към други места в паметта, където са неговите деца. Представяне:

```
template<typename T>
```

```
struct Node {
    T value;
    Node* left = nullptr;
    Node* right = nullptr;
}
```

Сложност на операциите - добавяне, премахване, намиране на елемент

- Зависи от баласираността

Операция	**Worst case**	**Average case**
търсене	*O(N)*	*Θ(logN)*
добавяне	*O(N)*	*Θ(logN)*
триене	*O(N)*	*Θ(logN)*
обхождане	*O(N)*	*Θ(N)*

Където N е броят на елементите в дървото.

Имплементация на двоично кореново дърво за търсене

В съответната папка.